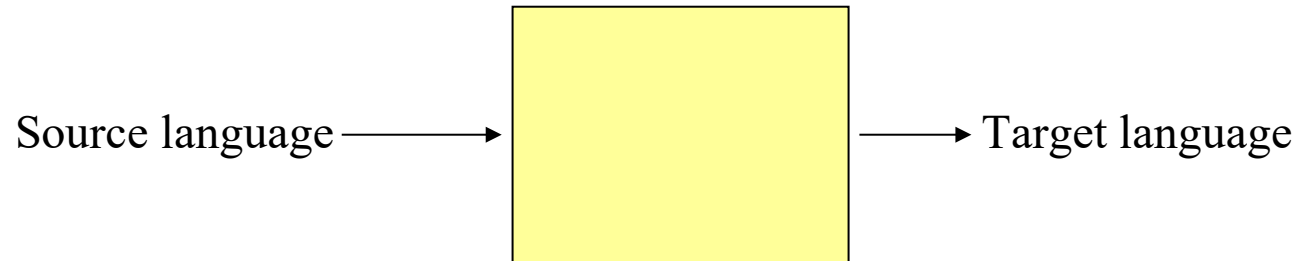


WHAT IS A COMPILER?

A program that reads a program written in one language and translates it into another language.



Traditionally, compilers go from high-level languages to low-level languages.

A compiler is a program that translates (or compiles) a program written in a **high-level** programming language (the source language) that is suitable for human programmers into the **low-level** machine language (target language) that is required by computers.

During this process, the compiler will also attempt to spot and report obvious programmer mistakes that are detected during the translation process.

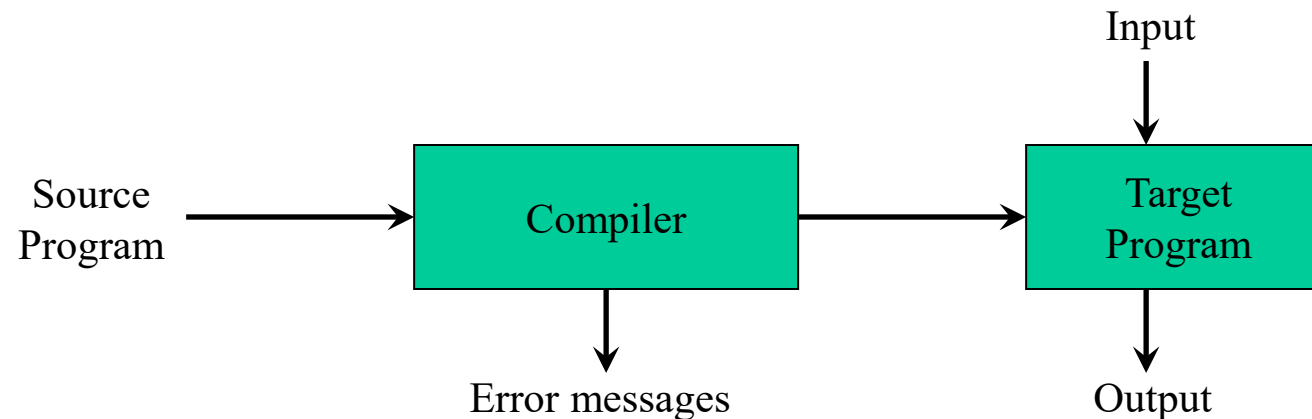
Why we use high-level language for programming?

Using a high-level language for programming has a large impact on how fast programs can be developed. The main reasons for this are:

1. Compared to machine language, the notation used by programming languages is closer to the way humans think about problems.
2. The compiler can spot some obvious programming mistakes.
3. Programs written in a high-level language tend to be shorter than equivalent programs written in machine language.
4. The same program can be compiled to many different machine languages and, hence, be brought to run on many different machines.

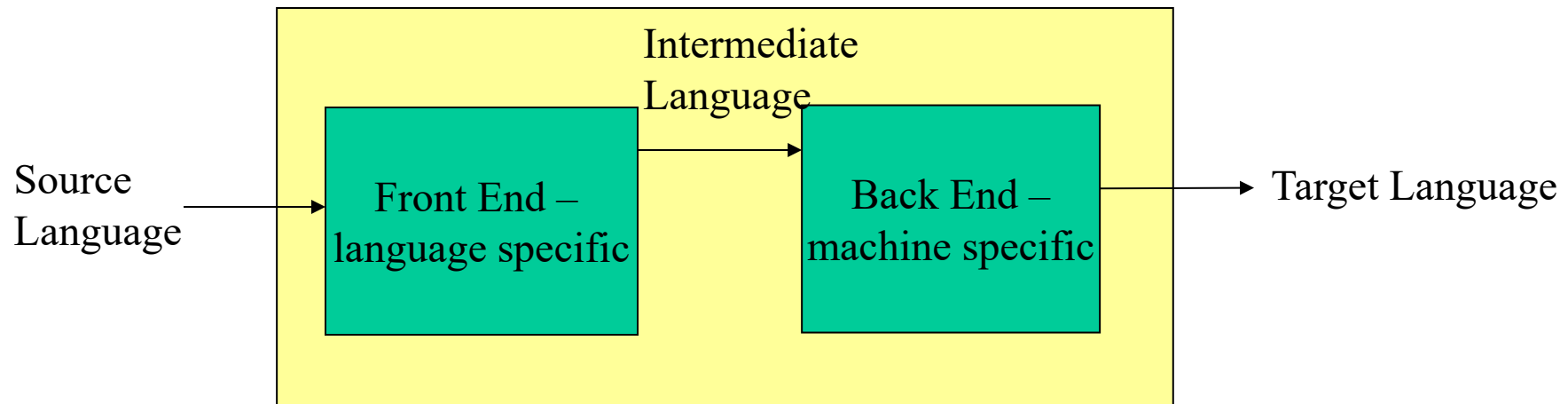
Compilers and Interpreters

- **“Compilation”**
 - Translation of a program written in a source language into a semantically equivalent program written in a target language



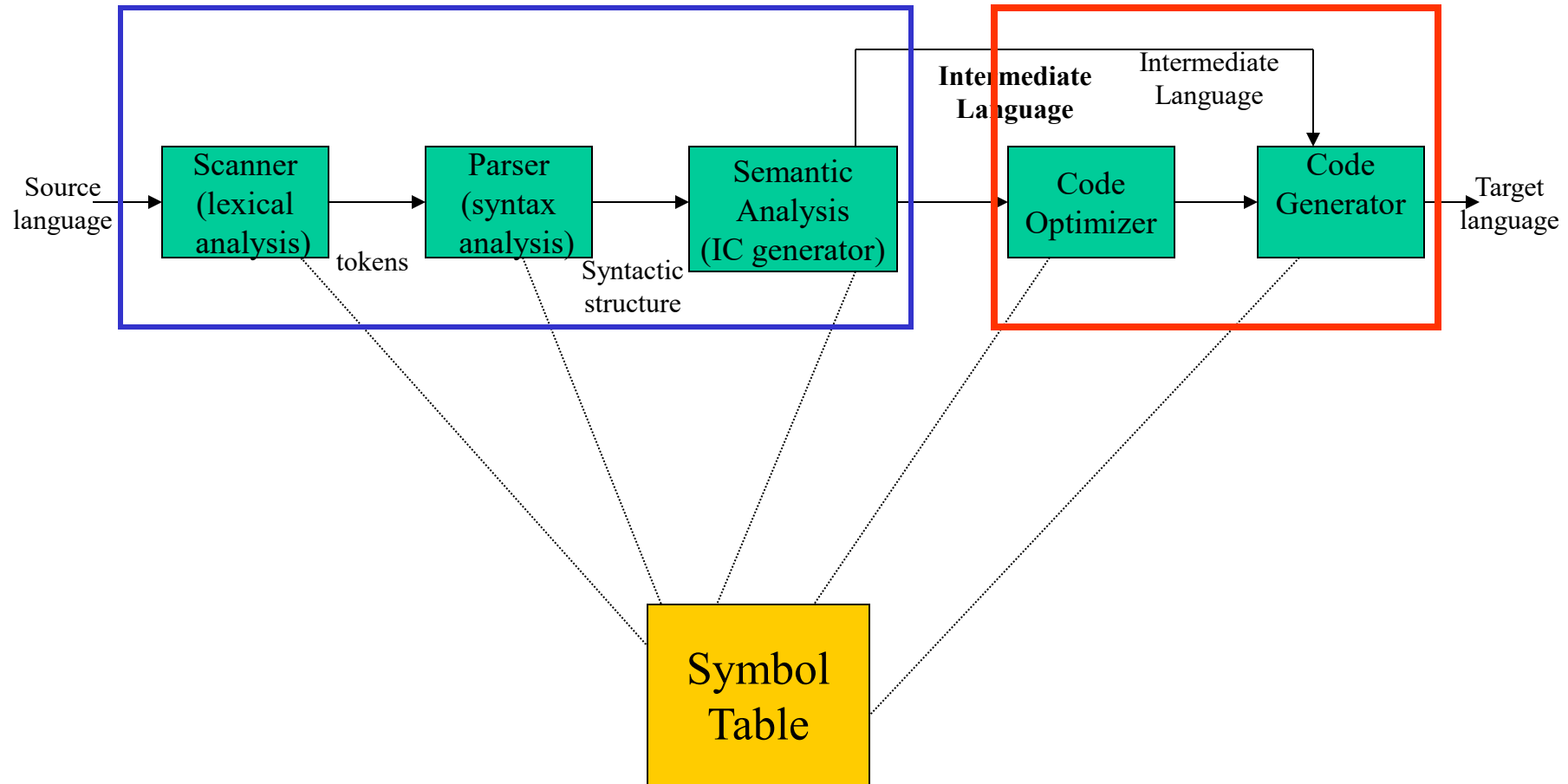
COMPILER ARCHITECTURE

In more detail:



- Separation of Concerns
- Retargeting

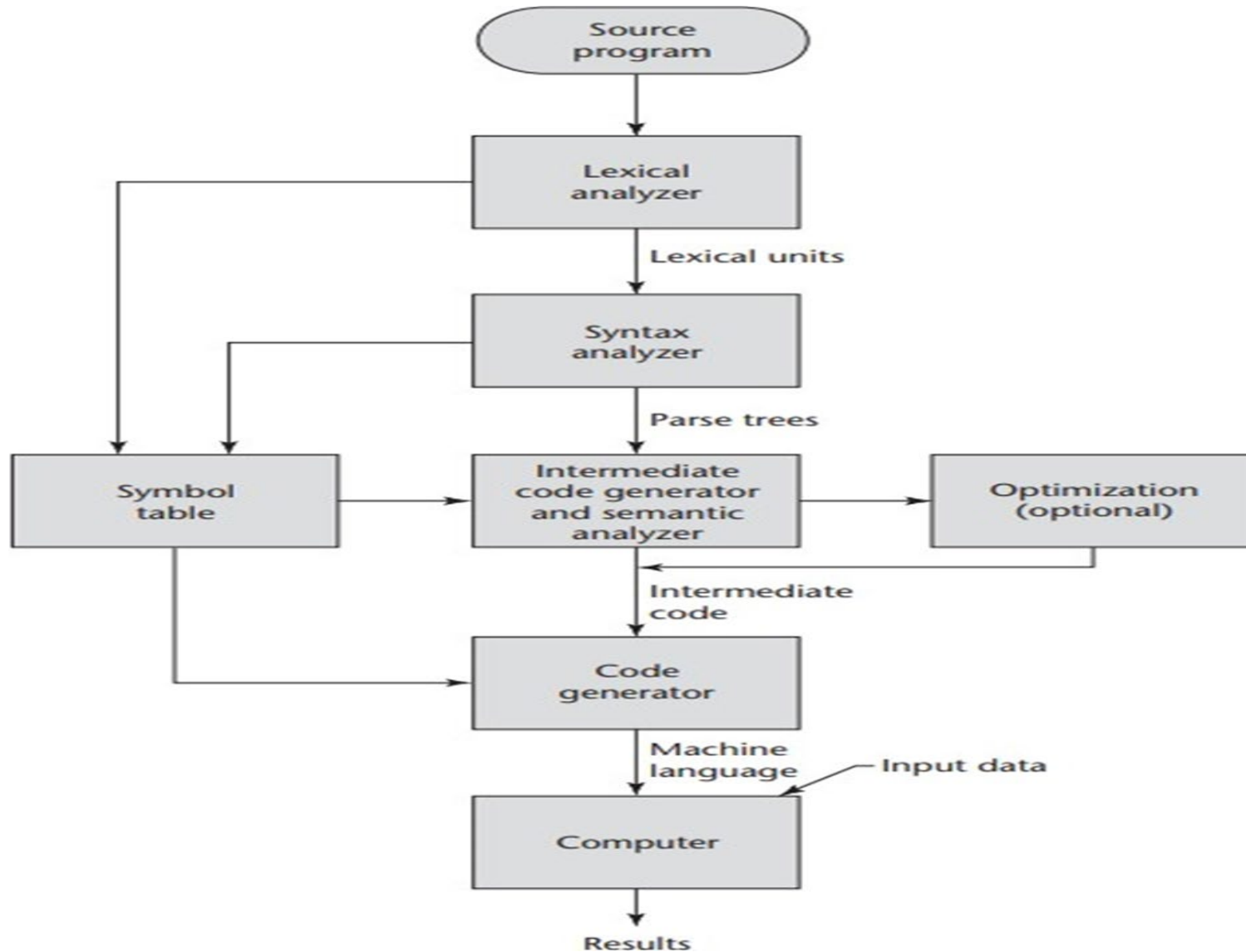
COMPILER ARCHITECTURE



COMPILATION PROCESS

The compilation process details the stages a source program goes through to produce machine-executable code. Below is a detailed explanation of each stage

Outline of the compilation process



1. Source Program

This is the input written in a high-level programming language (e.g., C, Java, or Python) that needs to be compiled into machine code.

2. Lexical Analyzer

- Breaks down the source program into **lexical units** (tokens), such as keywords, operators, identifiers, and literals.
- Tokens that represent the basic syntax elements of the source program.
- **Example:** The statement `int a = 5;` would be broken into tokens like `int`, `a`, `=`, `5`, and `;`.
- Detects lexical errors such as invalid characters in the source program.

Input: $\text{result} = a + b * c / d$

- Tokens:

‘result’, ‘=’, ‘a’, ‘+’, ‘b’, ‘*’, ‘c’, ‘/’, ‘d’

Identifiers

Operators

3. Syntax Analyzer

- Analyzes the structure of the program using the tokens from the lexical analyzer.
- A **parse tree**, which represents the hierarchical structure of the source code according to the grammar of the language.
- Identifies syntax errors, such as missing parentheses or incorrect statement structure.
- Ensures `if (a > b)` is correctly structured.

Input: result = a + b * c / d

Exp ::= Exp '+'

Exp

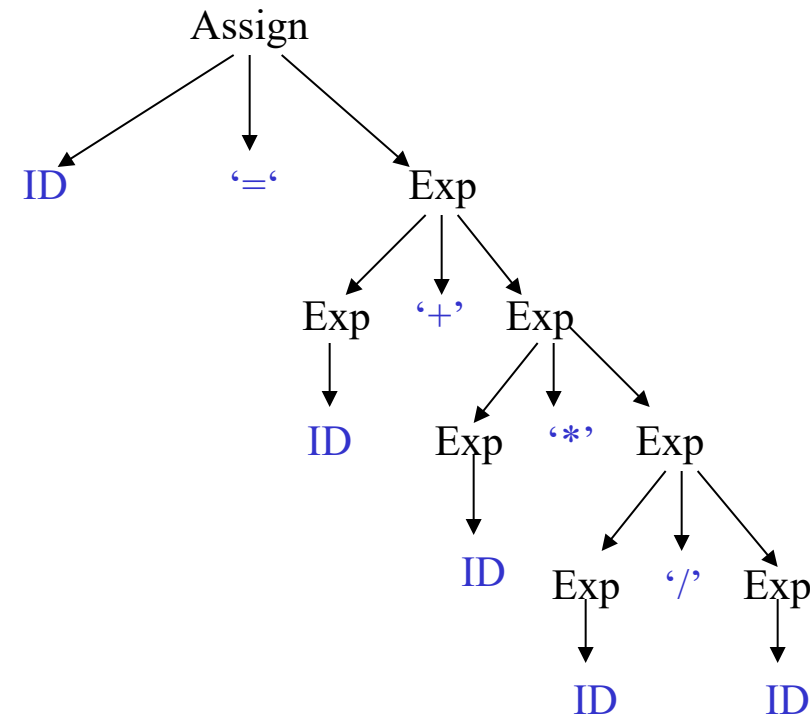
| Exp '-' Exp

| Exp '*' Exp

| Exp '/' Exp

| ID

Assign ::= ID '=' Exp



Input: $\text{result} = a + b * c / d$

- The expression part is broken down considering operator precedence:
Multiplication (*) and division (/) are evaluated before addition (+).
Division (/) has the same precedence as multiplication and is evaluated left to right
- The tree reflects operator precedence and associativity. It shows how a compiler breaks the expression into a syntax tree for further analysis. This structure helps generate intermediate code or perform semantic checks.

4. Symbol Table

- . Maintains a record of all identifiers (e.g., variables, functions) used in the program, along with their attributes such as type, scope, and memory location.
- . Updated and accessed throughout the compilation process to ensure consistency and resolve references.

5. Intermediate Code Generator and Semantic Analyzer

- **Semantic Analyzer:**

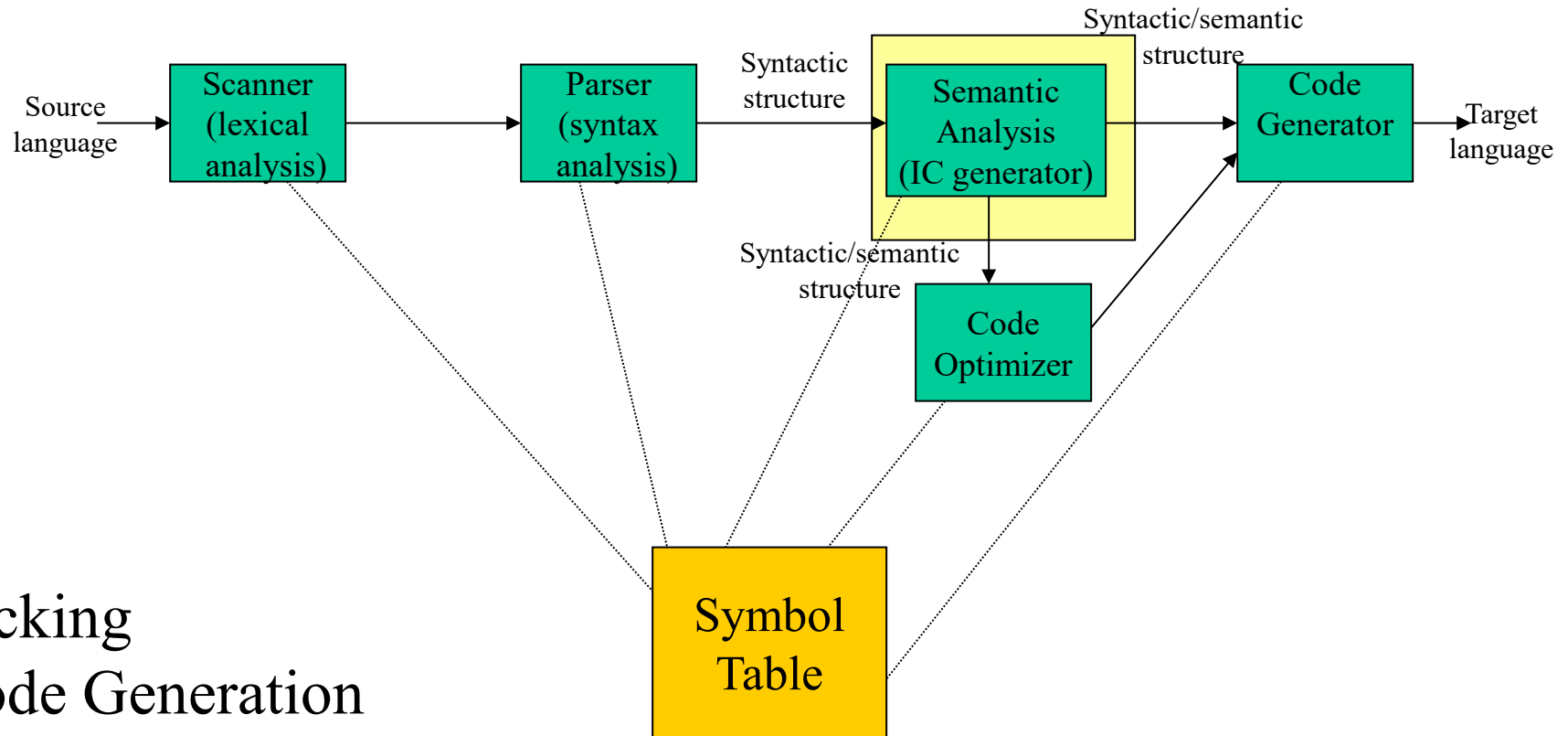
- Ensures that the program's logic and semantics are correct (e.g., type compatibility in assignments).
- Example: Verifies if adding a string and an integer is valid.

- **Intermediate Code Generator:**

- Produces an **intermediate representation (IR)** of the program, which is closer to machine code but still abstract.
- Example: Converts $a = b + c;$ into intermediate code like `ADD b, c -> a.`

- Intermediate code, simplifies optimization and code generation.

Semantic Analysis

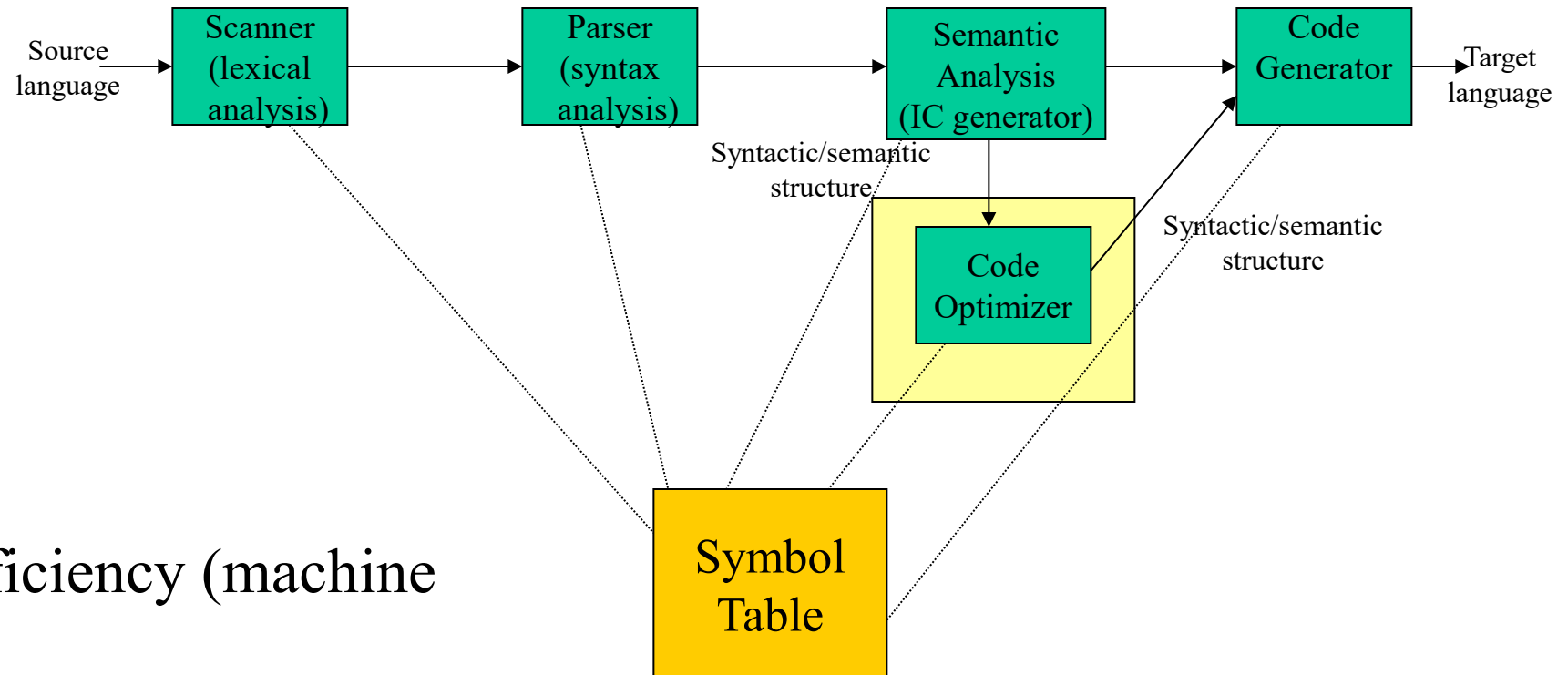


- “Meaning”
- Type/Error Checking
- Intermediate Code Generation
 - abstract machine

6. Optimization (Optional)

- Improves the efficiency of the intermediate code by:
 - Reducing redundant computations.
 - Minimizing resource usage (e.g., memory and CPU).
- **Example:** If the value of a variable does not change, redundant recalculations are removed.

Optimization

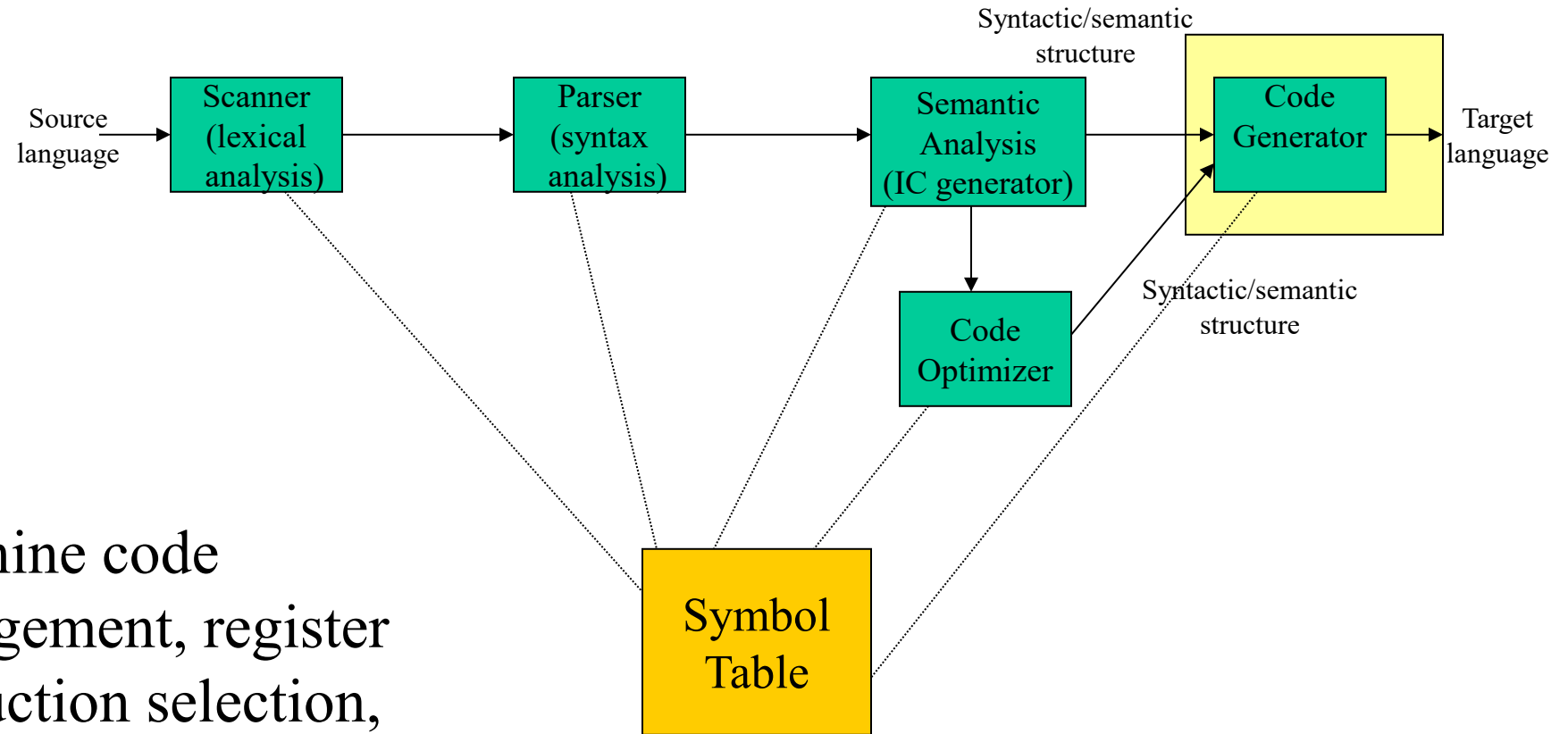


- Improving efficiency (machine independent)
- Finding optimal code is NP

7. Code Generator

- . Translates the optimized intermediate code into **machine language** or assembly code, specific to the target architecture.
- . Low-level machine code that can be executed by the computer's CPU.
- . **Example:** Converts intermediate operations into specific assembly instructions like MOV, ADD, and STORE.

Code Generation



- IC to real machine code
- Memory management, register allocation, instruction selection, instruction scheduling, ...

8. Computer

- . Machine code and user-provided inputs are executed on the hardware.
- . Produces the final result of the program, such as computations, file manipulations, or visual outputs.

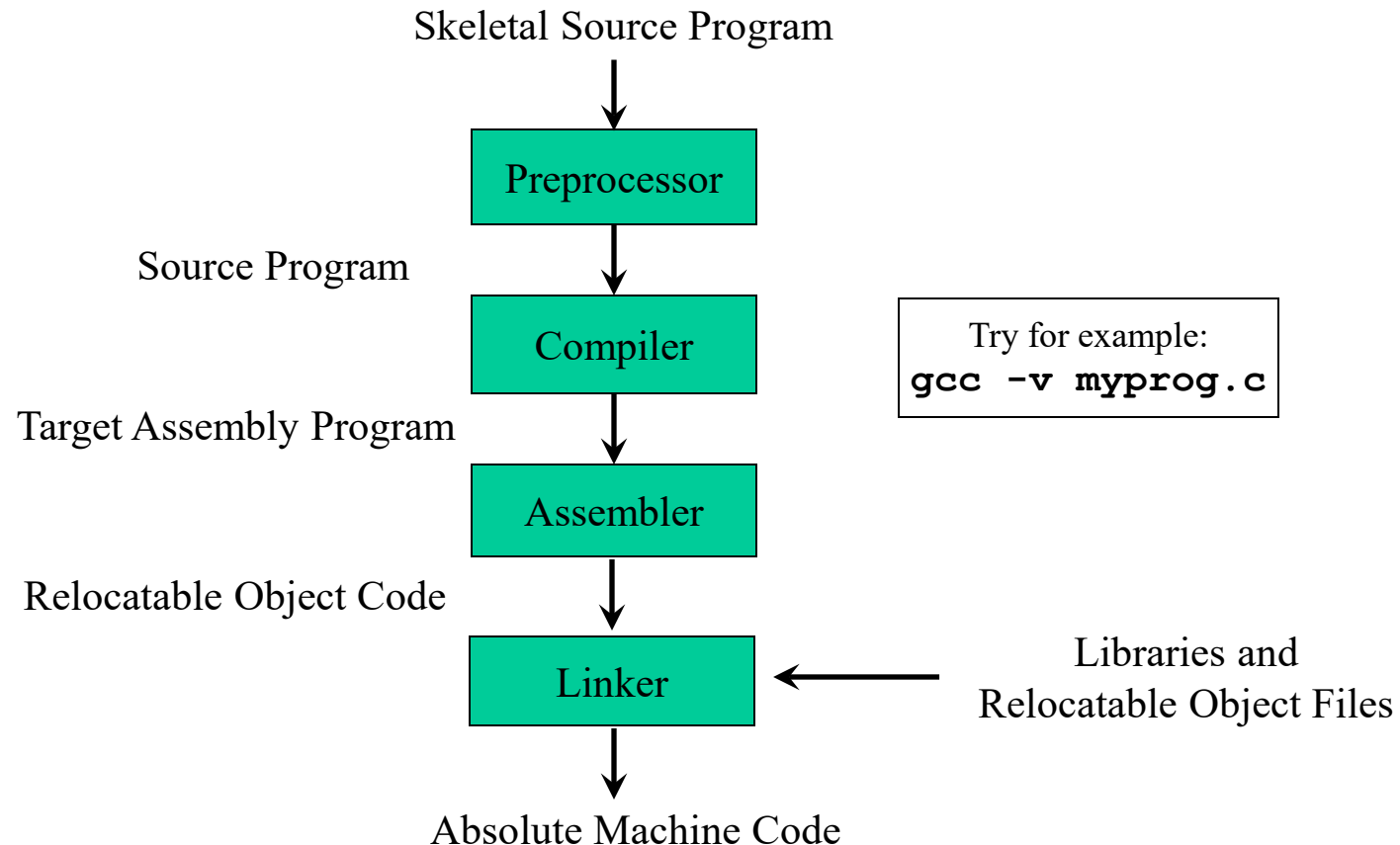
Summary of Error Detection and Handling:

- **Lexical Errors:** Detected during tokenization (e.g., invalid symbols).
- **Syntax Errors:** Identified by analyzing the parse tree structure.
- **Semantic Errors:** Found during the semantic analysis (e.g., type mismatches).
- **Runtime Errors:** Not addressed in the compilation process but handled during execution.

The Phases of a Compiler

Phase	Output	Sample
Programmer	Source string	A=B+C ;
Scanner (performs lexical analysis)	Token string	\A', \=', \B', \+', \C', \;' And <i>symbol table</i> for identifiers
Parser (performs syntax analysis based on the grammar of the programming language)	Parse tree or abstract syntax tree	<pre> ; = /\ A + /\ B C </pre>
Semantic analyzer (type checking, etc)	Parse tree or abstract syntax tree	
Intermediate code generator	Three-address code, quads, or RTL	<pre> int2fp B t1 + t1 C t2 := t2 A </pre>
Optimizer	Three-address code, quads, or RTL	<pre> int2fp B t1 + t1 #2.3 A </pre>
Code generator	Assembly code	<pre> MOVF #2.3,r1 ADDF2 r1,r2 MOVF r2,A </pre>
Peephole optimizer	Assembly code	<pre> ADDF2 #2.3,r2 MOVF r2,A </pre>

Preprocessors, Compilers, Assemblers, and Linkers



1. Preprocessor

- Input: **Skeletal source program** (e.g., #include, #define in C)
- Action: Handles directives like macro expansion, file inclusion, and conditional compilation.
- Output: A **complete source program** with all preprocessor directives resolved.
- Example tool: gcc -E myprog.c

2. Compiler

- Input: Preprocessed source code
- Action: Translates high-level code (like C, C++) into **assembly code**.
- Output: A **target assembly program**
- Example tool: `gcc -S myprog.c`

3. Assembler

- Input: Assembly code
- Action: Converts the assembly language into **machine code**, specifically a **relocatable object file**.
- Output: **Object code (.o/.obj)** that isn't fully ready to run
- Example tool: `as myprog.s -o myprog.o`

4. Linker

- Input: Object files + libraries
- Action: Combines your code with library functions, resolves references, and produces **absolute machine code**.
- Output: An **executable file** (e.g., .exe, a.out)
- Example tool: ld or invoked via gcc

Issues Driving Compiler Design

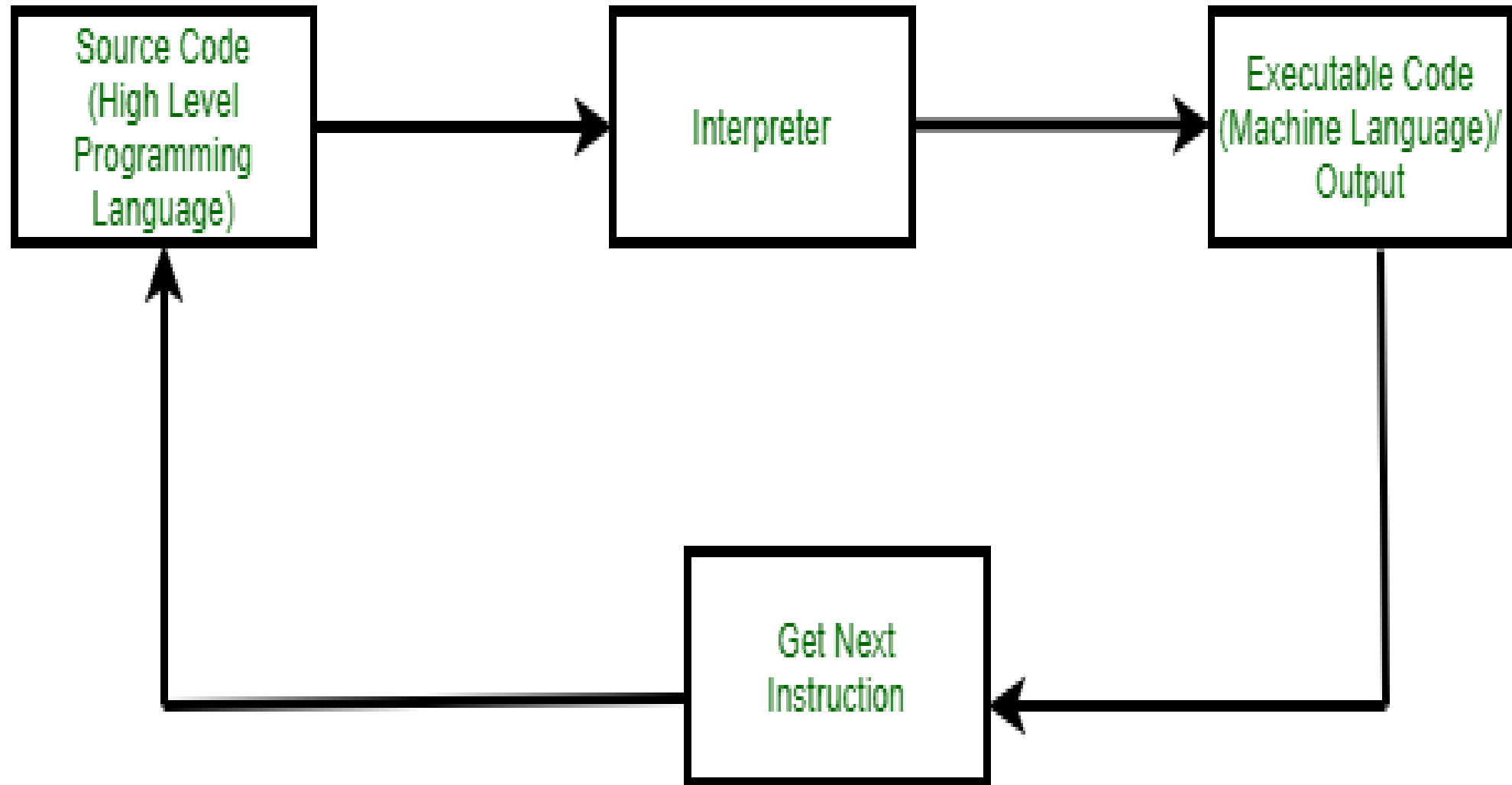
- Correctness
- Speed (runtime and compile time)
 - Degrees of optimization
 - Multiple passes
- Space
- Feedback to user
- Debugging

Why study compilers?

- Bring together:
 - Data structures & Algorithms
 - Formal Languages
 - Computer Architecture
- Influence:
 - Language Design
 - Architecture (influence is bi-directional)
- Techniques used influence other areas (program analysis, testing, ...)

INTRODUCTION TO INTERPRETATION

- . Interpretation involves executing a program without prior translation to machine code.
- . The **interpreter** acts as a virtual machine that processes high-level language statements directly.
- . Unlike compilation, which generates an executable file, the interpreter processes and evaluates source code dynamically at runtime.



Translation Process in Interpretation

a. Read Phase (Input Analysis)

- The interpreter **reads a statement** or expression from the source program.
- The source code is typically parsed into an **internal representation**, such as an abstract syntax tree (AST) or another intermediate structure, to facilitate evaluation.
- Example:
 - $x = 5 + 3$
 - This statement would be broken down into:
 - A variable x .
 - An assignment operator $=$.
 - An expression $5 + 3$ to evaluate.

b. Evaluation Phase

- The interpreter evaluates the internal form of the statement.
- The interpreter traverses the internal representation and computes the result of expressions, performs assignments, or calls functions.
- This phase corresponds to the **execute cycle** in traditional fetch-execute operations of hardware.
- Example:
 - Evaluating $5 + 3$ computes the result 8.
 - The interpreter then binds the result to the variable `x`.

c. Print Phase

- . If the statement or expression produces a visible output (e.g., a print statement or a result in a REPL), the interpreter displays it to the user.
- . Example:

```
print(x)
```

- Output: 8.

d. Looping (Repeat)

- . After processing one statement, the interpreter **returns to the input phase** and reads the next statement or expression.
- . This cycle (read $\rightarrow$ evaluate $\rightarrow$ print $\rightarrow$ loop) continues until:
 - There are no more statements.
 - A specific exit condition (e.g., an `exit` or `quit` command) is encountered.
- . This is commonly referred to as the **Read-Eval-Print Loop (REPL)**.

Example of Read-Eval-Print Loop (REPL)

- A Python interpreter follows this process:
 - **Read:** The user enters `x = 10 + 5`.
 - **Evaluate:**
 - The interpreter parses the input.
 - It computes the expression `10 + 5` to get `15`.
 - Assigns the result to `x`.
 - **Print:** If it is an interactive session (e.g., REPL), the result or status is shown.
 - **Loop:** The interpreter waits for the next statement.

Comparison with Compilation

- **Compilation:**

- Translates the entire source program into machine code before execution.
- Faster execution after compilation.
- Errors are detected during compilation, not at runtime.

- **Interpretation:**

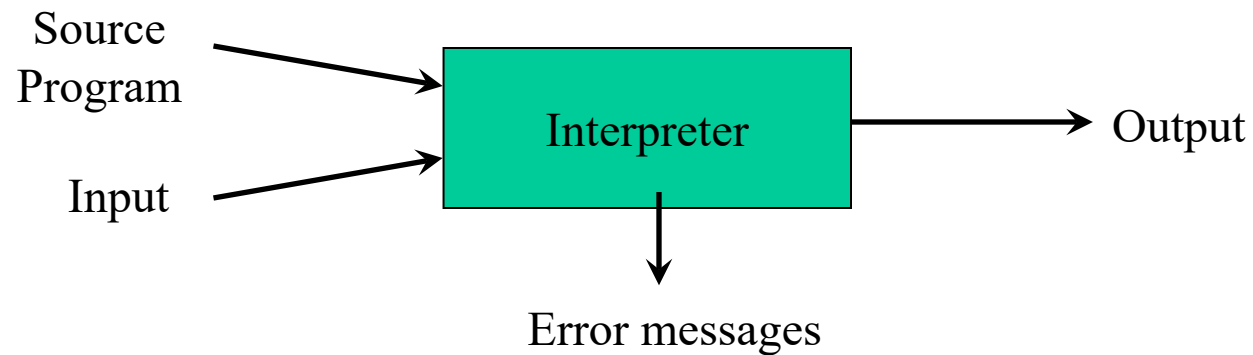
- Executes statements line by line without producing machine code.
- Slower but allows immediate error detection and debugging.

Applications of Interpretation

- . Scripting languages: **JavaScript**, **PHP**, and **Python**.
- . Prototyping and dynamic environments: **MATLAB** or interactive shells.
- . Web development: Server-side interpretation for web scripts.

Compilers and Interpreters (cont'd)

- **“Interpretation”**
 - Performing the operations implied by the source program



Compiler vs. Interpreter

Compiler

- Takes Entire program as input
- It is **Faster**
- intermediate object code is **generated**.
- Required **more** memory Due to intermediate object code
- Program **not need compile** every time
- **Errors** are displayed after **entire program** is checked.
- **Debugging** is comparatively **hard**.
- Ex: C, C++.

Interpreter

- Take single instruction as input
- It is **Slower**
- **No** intermediate code is **generated**
- Required **less** memory As no intermediate code is generated
- **Every time** higher level program is converted into lower level program.
- **Errors** are displayed **for every instruction** interpreted.
- **Debugging** is **easy**.
- Ex: python, Ruby, basic.

Hybrid compiler

Compilation and interpretation may be combined to implement a programming language:

The compiler may produce intermediate-level code, which is then interpreted rather than compiled to machine code.

Ex: java

Source Program

Translator
(Compiler)

Intermediate
Program

Input

Virtual
machine
(Interpreter)

Output

The Analysis-Synthesis Model of Compilation

- There are two parts to compilation:
 - **Analysis** determines the operations implied by the source program, which are recorded in a tree structure
 - **Synthesis** takes the tree structure and translates the operations therein into the target program

Other Tools that Use the Analysis-Synthesis Model

- Editors (syntax highlighting)
- Pretty printers (e.g. doxygen)
- Static checkers (e.g. lint and splint)
- Interpreters
- Text formatters (e.g. TeX and LaTeX)
- Silicon compilers (e.g. VHDL)
- Query interpreters/compilers (Databases)

Compiler-Construction Tools

1. **Parser generators** that automatically produce syntax analyzers from a grammatical description of a programming language.
2. **Scanner generators** that produce lexical analyzers from a regular-expression description of the tokens of a language.
3. **Syntax-directed translation engines** that produce collections of routines for walking a parse tree and generating intermediate code.

Compiler-Construction Tools

4. Code-generator generators that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.

5. Data-flow analysis engines that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.

6. Compiler-construction toolkits that provide an integrated set of routines for constructing various phases of a compiler.

Why learn about compilers?

- It is considered a topic that you should know in order to be “well-cultured” in computer science.
- A good craftsman should know his tools, and **compilers are important tools for programmers** and computer scientists.
- The techniques used for constructing a compiler **are useful for other purposes** as well.
- There is a good chance that a programmer or computer scientist will need **to write a compiler or interpreter for a domain-specific language**.

The Evolution of Programming Language

Classification by generation

- ❑ **First-generation languages:** **machine** languages
- ❑ **Second-generation :** **assembly** languages
- ❑ **Third-generation :** **higher-level** languages like Fortran, Cobol, Lisp, C, C++, C#, and Java.
- ❑ **Fourth-generation languages:** languages **designed for specific applications** like NOMAD for report generation, SQL for database queries, and Postscript for text formatting.
- ❑ **fifth-generation language** has been **applied to logic- and constraint-** based languages like Prolog and OPS5.

Impacts on Compilers

- The advances in programming languages placed new demands on compiler writers.
- Compiler writers would take maximal advantage of the new hardware capabilities.
- Good software-engineering techniques are essential for creating and evolving modern language processors.

The Grouping of Phases

- **Compiler** front and back ends:
 - Analysis (*machine independent* front end)
 - Synthesis (*machine dependent* back end)
- **Passes**
 - A collection of phases may be repeated only once (*single pass*) or multiple times (*multi pass*)
 - Single pass: usually requires everything to be defined before being used in source program
 - Multi pass: compiler may have to keep entire program representation in memory

Compiler-Construction Tools

Software development tools are available to implement one or more compiler phases:

- Scanner generators
- Parser generators
- Syntax-directed translation engines
- Automatic code generators
- Data-flow engines

Summary

- **Language Processors:** An integrated software development environment includes many different kinds of language processors such as compilers, interpreters, assemblers, linkers, loaders, debuggers, profilers.
- **Compiler Phases:** A compiler operates as a sequence of phases, each of which transforms the source program from one intermediate representation to another.
 - Lexical Analyzer
 - Syntax Analyzer
 - Semantic Analyzer
 - Intermediate Code Generator
 - Machine-Independent Code Optimizer
 - Code Generator
 - Machine-Dependent Code Optimizer
- **Machine and Assembly Languages:** Machine languages were the first generation programming languages, followed by assembly languages. Programming in these languages was **time consuming** and **error prone**.